

Floyd’s Tortoise and Hare

Huiri Chen

June 2025

I Introduction

In the field of computer science, particularly within the study of data structures and algorithms, efficiently detecting cycles in data sequences is a fundamental problem. Floyd’s Tortoise and Hare algorithm, also known as the cycle detection algorithm, provides an elegant and highly optimized solution to this problem. Developed by Robert W. Floyd in 1967, the algorithm is celebrated for its efficiency, requiring only constant space $\mathcal{O}(1)$ and linear time $\mathcal{O}(n)$.

The central idea involves using two pointers that traverse a sequence at different speeds—a slow-moving “*tortoise*” that takes one step at a time, and a fast-moving “*hare*” that takes two steps at a time. If the sequence contains a cycle, the faster hare will eventually overlap with the slower tortoise from behind. This approach is widely used to detect cycles in linked lists, analyze functional graph traversals, and evaluate pseudo-random number generators.

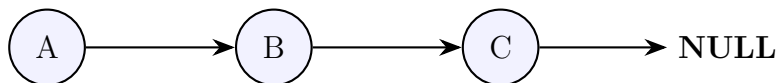
In this document, we will explore the theory behind Floyd’s Tortoise and Hare algorithm, visually walk through its mechanics, and mathematically prove how it locates the exact starting node of a cycle.

II Singly Linked Lists and Cycles

1 Standard Linked List

A singly linked list is a linear collection of data elements where the order is dictated by pointers rather than contiguous physical placement in memory. Each element, called a node, contains data and a reference (or pointer) to the next node in the sequence. The list ends when a node points to a null reference.

Visualization:

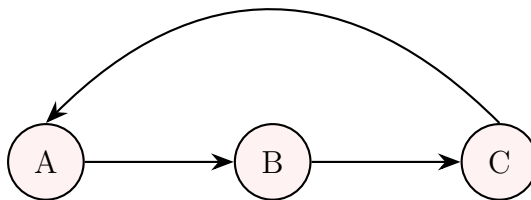


2 Defining a Cycle

Consider a scenario where node A points to node B, node B points to node C, and node C points back to node A. Traversing this list traps you in an infinite loop: $A \rightarrow B \rightarrow C \rightarrow A \rightarrow \dots$. You will never reach a `nullptr`.

In general, a cycle exists if a traversal path starts at a node N_i and, by continuously following the pointers, eventually returns to N_i .

Visualization of a Cycle:



III The Algorithm

The algorithm executes in two distinct phases: determining if a cycle exists, and locating the entry point of the cycle.

Phase 1: Detection We initialize two pointers, *tortoise* and *hare*, both starting at the head of the linked list. As we iterate through the list:

- *tortoise* advances one node per step.
- *hare* advances two nodes per step.

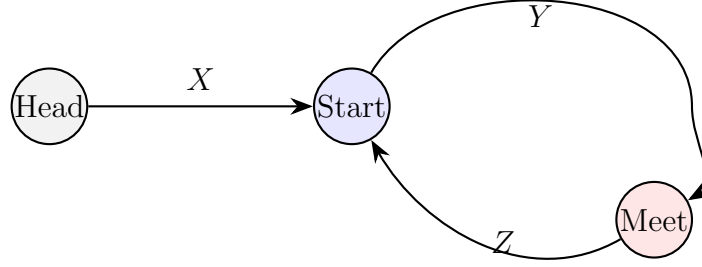
If *hare* (or its next pointer) reaches `nullptr`, the list terminates linearly, confirming that **no cycle exists**. However, if *hare* and *tortoise* eventually point to the exact same node, a cycle is guaranteed.

Phase 2: Finding the Cycle Start Once they meet, we leave one pointer at the meeting node and move the other pointer back to the head of the list. We then advance *both* pointers at the same speed (one step at a time). The node where they collide for the second time is the exact beginning of the cycle.

IV Theoretical Background and Proof

To understand why Phase 2 works, we must analyze the distances traveled by both pointers at the moment they first meet.

Visualization of the Distances:



Let us define the following distance variables:

- X : The distance (number of nodes) from the **Head** to the **Start** of the cycle.
- Y : The distance from the **Start** of the cycle to the **Meeting** point.
- Z : The remaining distance from the **Meeting** point back to the **Start** of the cycle.

From this, we know the total length of the cycle is $C = Y + Z$.

When the two pointers meet, we can express the total distance traveled by each:

- Distance traveled by *tortoise* $= X + Y$
- Distance traveled by *hare* $= X + Y + kC$ (where $k \geq 1$ is the number of times the hare looped around the cycle)

Because the hare moves exactly twice as fast as the tortoise, the distance covered by the hare is twice the distance covered by the tortoise:

$$2 \cdot (X + Y) = X + Y + kC \quad (1)$$

By subtracting $X + Y$ from both sides, we simplify this to:

$$X + Y = kC \quad (2)$$

Now, we want to solve for X (the distance from the head to the cycle start):

$$X = kC - Y \tag{3}$$

We can break down kC into $(k - 1)C + C$. Substituting this in gives:

$$X = (k - 1)C + (C - Y) \tag{4}$$

Looking at our definitions, the total cycle length C minus the distance Y is exactly the remaining distance Z ($C - Y = Z$). Therefore:

$$X = (k - 1)C + Z \tag{5}$$

Conclusion:

Equation 5 fundamentally proves the algorithm. It states that the distance from the Head to the Start (X) is equal to the distance from the Meeting point to the Start (Z), plus some number of full loops around the cycle $((k - 1)C)$.

Therefore, if we place one pointer at the Head (which needs to travel X steps to reach the Start) and leave one pointer at the Meeting point (which needs to travel Z steps to reach the Start), advancing them both by one step at a time guarantees they will collide exactly at the starting node of the cycle.